How does ***CDC (Change Data Capture) work in 5 DBs.***

(Postgres, MySQL, DynamoDB, Cassadra, MongoDB)

Chandra Shekhar Joshi

Engineering career coach

01

If you're Sr+ Engineer, and preparing for FAANG+ System Design interview...

It’s not enough to just know that CDC exists, you need to understand how CDC actually works to use it confidently in your Design, and to discuss with interviewer.

02

What is CDC?

CDC is a pattern that captures every change, INSERT, UPDATE, DELETE, in your database as it happens, and streams it as a real-time event.

Instead of querying “what changed?” every 5 seconds…

CDC gives you a live feed of all changes, like a version controlled stream of diffs for your database.

It’s like subscribing to a “new arrivals” newsletter from your database, rather than checking the shelves every morning.

Or Git for your data, where each change is versioned, ordered, and broadcast.

04

Why it matters?

It lets you sync downstream systems (search index, cache, warehouse) without scanning whole tables.

It enables real-time features (notifications, analytics dashboards).

It reduces load on primary DB because you don’t run heavy polling queries.

Read my last post on ***When to use CDC in System Design interviews*** to know about why and when.

06

How CDC generally works

Step 1: Snapshot - Take a point-in-time copy of existing data to get the starting state.

Step 2: Bookmark - Record the exact position in the database log at snapshot start.

Step 3: Stream - After snapshot, stream every change from that bookmark forward.

Step 4: Acknowledge + Persist Position - Save progress so you can resume after crash/restart.

08

Common building blocks

Database log or stream: WAL, binlog, oplog, Streams, cdc_raw files

Decoder / connector: Converts log bytes into structured events (JSON/Avro)

Consumer: Reads events and applies them to target systems (search, cache, analytics)

Bookmark: Pointer (LSN / GTID / token) so you never miss or double-process data

10

PostgreSQL (WAL + Logical Decoding)

Core idea

PostgreSQL writes every change to WAL (Write-Ahead Log) first.

CDC taps into this WAL, so you are not adding a heavy new process.

This keeps performance impact low.

From bytes to events

The raw WAL is just a stream of bytes, not human-readable.

To use it, PostgreSQL's built-in feature called logical decoding turns the WAL into a structured stream of events.

Most modern CDC tools (like Debezium) use this feature directly.

No extra plugin is required in standard PostgreSQL, but you do need to enable logical replication and set *wal_level=logical*.

Snapshot → LSN handoff → Stream

Step 1: Consistent snapshot.

- Start by copying the current data for the tables you want.
- Take a brief lock to mark a point-in-time view so ongoing transactions don’t corrupt the snapshot.
- Use this to seed downstream systems (search index, warehouse, etc.).

(Continued in next page...)

Snapshot → LSN handoff → Stream

Step 2: Record the LSN.

- At the moment the snapshot starts, record the LSN (Log Sequence Number) from the WAL.
- After the snapshot finishes, tell PostgreSQL: “Send all changes since that LSN.”
- This prevents gaps or duplicates during the handoff.

Step 3: Stream events.

- Logical decoding sends one event per committed row change with:
- op: c/u/d,
- before (for update/delete), after (for insert/update),
- source metadata (DB, schema, table, LSN).

Reliability using Replication Slot

For reliability, PostgreSQL uses a Logical Replication Slot.

This replication slot is a durable bookmark that tracks exactly how far your connector has read.

It also instructs PostgreSQL not to delete WAL until the connector confirms it has read that part.

The replication slot survives crashes and restarts, so you can resume reading from the exact point you left off.

Watch disk usage and monitor replication slot

If the consumer stops or falls behind, PostgreSQL will keep WAL files on disk because the replication slot says they are still needed.

If this continues, the WAL files can fill the entire disk and cause a database outage.

You must monitor replication slot lag and disk usage regularly.

Setup: Easy

Moderate with tools like Debezium. The main challenge is keeping an eye on replication slot health to avoid disk issues.

17

MySQL (Binary Log + GTIDs)

Core idea

MySQL writes every data change to the binary log (binlog) so replicas can stay in sync.

CDC simply reads the binlog, so you are using what MySQL already produces.

To capture exact data changes, you must set *binlog_format=ROW*.

In this format, MySQL records the actual before-and-after values of rows, not just the SQL query that caused the change.

Example: Instead of logging UPDATE users SET last_login = NOW(), it logs “row 123 changed last_login from NULL to 2025-09-21T06:00:00”, which is what you need for CDC.

Snapshot → position/GTID → stream

Step 1: Consistent snapshot.

- Connector takes a brief global read lock (FLUSH TABLES WITH READ LOCK) to freeze the database just long enough to:
  - Capture the schema
  - Record the current binlog position
- The lock is then released, and the connector runs a SELECT * FROM table to copy existing rows.

Step 2: Save the bookmark

- Bookmark = either binlog coordinates (file + position) or GTID set (preferred).
- GTID = Global Transaction ID, works across failovers, unlike file+position.

(Continued in next page...)

Snapshot → position/GTID → stream

Step 3: Start streaming

- The connector now acts like a replica and receives row-level change events from MySQL.
- Each event includes:
  - Operation type (c/u/d)
  - Before + after values of the row
  - Table, schema, database name
  - GTID and timestamp for ordering

Reliability and setup

MySQL does not hold logs for you like PostgreSQL does.

The connector must save its last processed binlog position or GTID somewhere reliable (disk, database, etc.).

MySQL deletes old binlog files automatically after expire_logs_days.

If the connector is down longer than that, the logs are gone → you must start over with a new snapshot.

(Continued in next page...)

Reliability and setup

ROW format is required (STATEMENT/MIXED are unsafe).

GTIDs recommended for production (simpler failover resume).

Setup: Easy to Moderate.

The main challenge is ensuring connector state is durable and binlog retention is long enough to cover possible downtime.

23

DynamoDB (DynamoDB Streams)

Core Idea

DynamoDB has a built-in feature called Streams.

When enabled, it records every item change (INSERT, MODIFY, REMOVE) in a time-ordered stream.

You choose what to capture:

- Only keys,
- New values,
- Old values,
- Or both before and after images (most complete).

Initial load + enabling stream

Step 1: Seed existing data (if needed)

- Export the whole table (via AWS Glue or a parallel scan) to build the starting state.
- Enable the stream first so you don't miss changes during the export.

Step 2: Enable Streams in table settings and pick your view type.

Reading the stream

Option 1: Lambda consumer

- Easiest way. AWS handles polling, batching, retries.
- Your Lambda function just processes the records.

Option 2: Custom consumer (SDK/KCL)

- More work, but full control over leases, checkpoints, scaling, and error handling.

Retention and ordering

Stream records last for 24 hours only.

If your consumer is down longer than that, the data is gone forever.

Ordering is guaranteed only within a shard (per partition key).

Setup: Very easy.
The main risk is consumer downtime beyond 24h.

Cassandra (Node-local CDC logs)

Core idea

Set table property WITH cdc = true.

Each node writes incoming changes for that table to its local cdc_raw directory.

There is no single central stream like in Postgres or MySQL.

Building a working pipeline

You must deploy a CDC agent on every node.

Each agent:

- Watches its local cdc_raw directory
- Reads new log files
- Parses raw mutation data
- Sends structured events (JSON/Avro) to a central message bus (usually Kafka)

The “stream” is assembled in Kafka by aggregating events from all node agents.

Downstream systems read from Kafka.

Complexity and risks

High operational overhead: must manage agents across every node.

No global order or deduplication built-in → your consumer must handle it.

No transaction boundaries → you have to infer them yourself.

If an agent lags, cdc_raw fills up and Cassandra will eventually reject writes.

Setup: Enabling CDC is easy. Running a production pipeline is hard.

Many teams prefer app-level event emission (Transactional Outbox pattern) instead.

MongoDB (Change Streams)

Core Idea

MongoDB has a Change Streams API built on the oplog (used for replication).

Requires a replica set or sharded cluster (not available on standalone).

Initial sync + opening the stream

Step 1: Initial sync if downstream needs full data. Query all documents first.

- Record a start time or logical timestamp.

Step 2: Open a change stream cursor using the driver (something like this)

- const changeStream = myCollection.watch();
- The cursor stays open and pushes events in real time.

Event structure + resume tokens

Each event contains:

- operationType (insert, update, delete)
- fullDocument (after image, if configured)
- fullDocumentBeforeChange (before image, if enabled)
- documentKey (_id)
- _id resume token

Save the resume token after processing each event.

If your app crashes, reopen the stream with resumeAfter(lastToken) to continue exactly where you left off.

Oplog rollover + setup

The oplog is a fixed-size capped collection.

If your app is down too long, older entries are overwritten → must re-sync.

For updates, configure before/after images to get full row state.

Setup: Moderate.
Main job is application code to store resume tokens and monitor lag.

Summary

Bookmarks and risk across databases

PostgreSQL: Replication slot holds WAL → great reliability, but watch disk.

MySQL: Connector stores position; retention must cover downtime.

DynamoDB: 24h retention is fixed → must consume continuously.

Cassandra: Agents + Kafka aggregation → heavy ops, dedupe/order manually.

MongoDB: Resume token + oplog size; re-sync if oplog rolls over.

Common patterns across databases

Snapshot first,

Record precise bookmark (LSN / GTID / token),

Stream changes after that bookmark,

Persist progress to resume safely.

While saying “I will use CDC” is good, but knowing how it works and what kind of challenges each database has in using CDC is super important for Sr/Staff roles.

This is the second post in CDC series.

Repost and follow for next post in CDC series.

Need help with mid-senior level SDE, EM interview prep?

DM me COACH